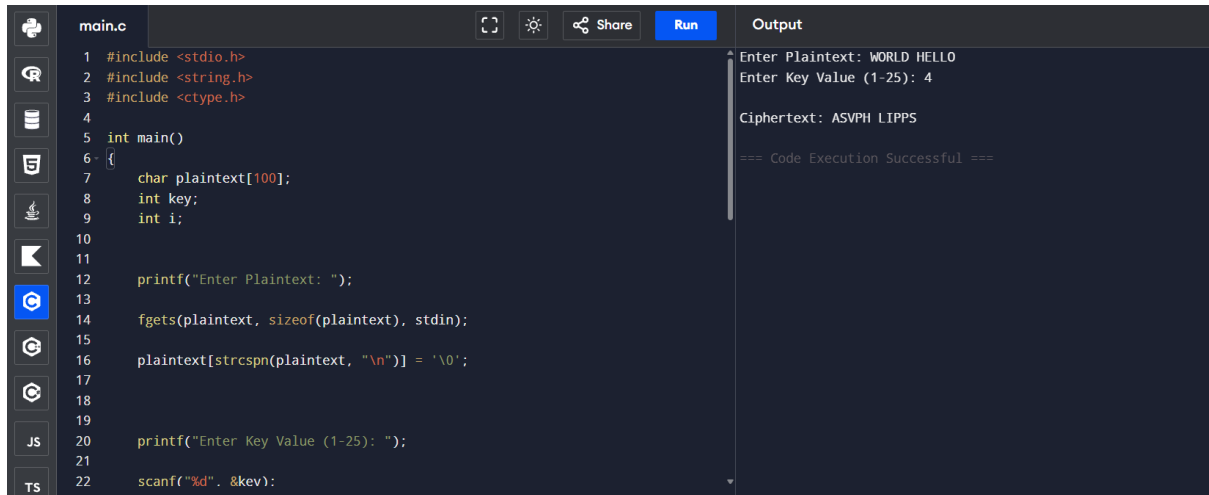


LAB PROGRAMS (1-10)

1) Caesar Cipher



```
main.c
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 int main()
6 {
7     char plaintext[100];
8     int key;
9     int i;
10
11     printf("Enter Plaintext: ");
12     fgets(plaintext, sizeof(plaintext), stdin);
13     plaintext[strcspn(plaintext, "\n")] = '\0';
14
15     printf("Enter Key Value (1-25): ");
16     scanf("%d", &key);
17
18
19
20
21
22
```

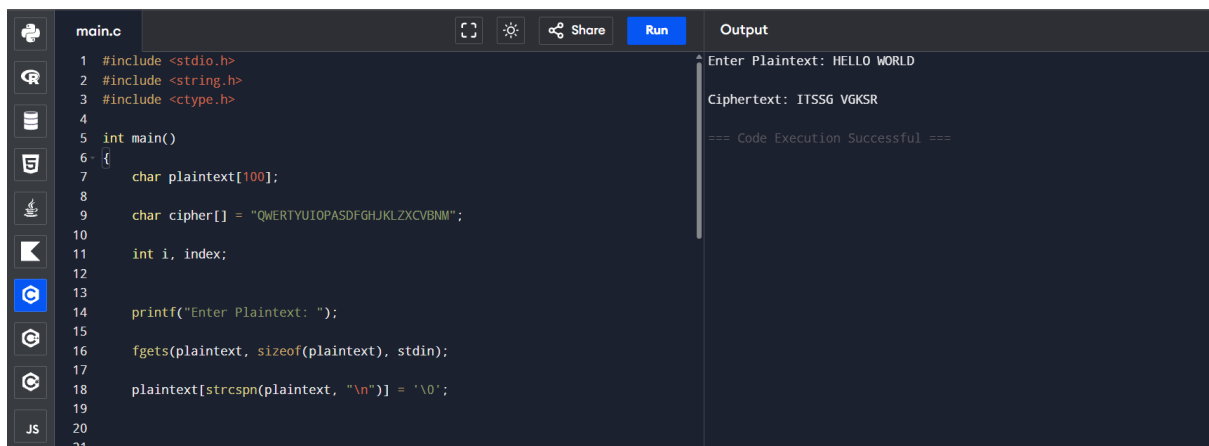
Output

```
Enter Plaintext: WORLD HELLO
Enter Key Value (1-25): 4

Ciphertext: ASVPH LIPPS

=== Code Execution Successful ===
```

2) Monoalphabetic Substitution Cipher



```
main.c
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 int main()
6 {
7     char plaintext[100];
8     char cipher[] = "QWERTYUIOPASDFGHJKLZXCVBNM";
9     int i, index;
10
11     printf("Enter Plaintext: ");
12     fgets(plaintext, sizeof(plaintext), stdin);
13     plaintext[strcspn(plaintext, "\n")] = '\0';
14
15
16
17
18
19
20
21
22
```

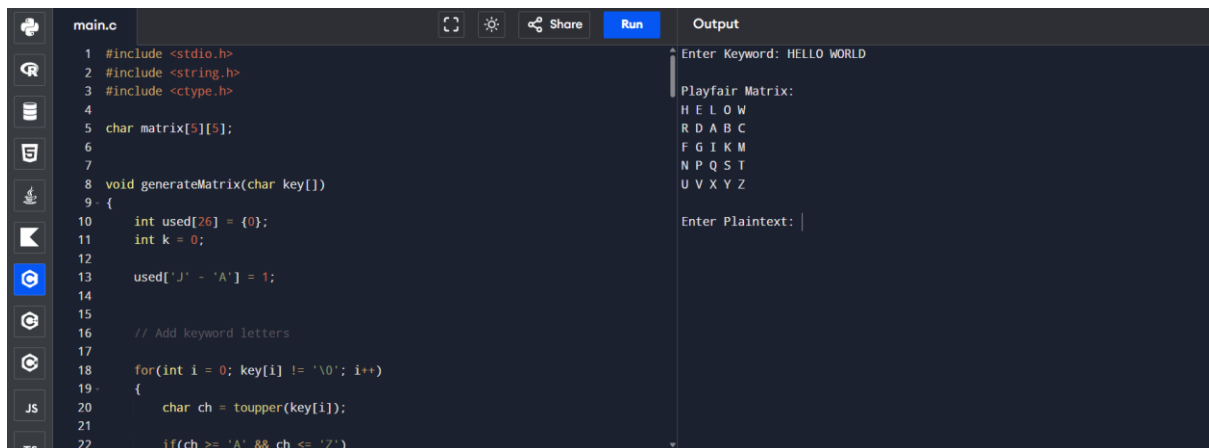
Output

```
Enter Plaintext: HELLO WORLD

Ciphertext: ITSSG VGKSR

=== Code Execution Successful ===
```

3) Playfair Cipher Algorithm



```
main.c
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 char matrix[5][5];
6
7 void generateMatrix(char key[])
8 {
9     int used[26] = {0};
10     int k = 0;
11
12     used['J' - 'A'] = 1;
13
14     // Add keyword letters
15     for(int i = 0; key[i] != '\0'; i++)
16     {
17         char ch = toupper(key[i]);
18         if(ch >= 'A' && ch <= 'Z')
19
20
21
22
```

Output

```
Enter Keyword: HELLO WORLD

Playfair Matrix:
H E L O W
R D A B C
F G I K M
N P Q S T
U V X Y Z

Enter Plaintext: |
```

4) Polyalphabetic Substitution Cipher (Vigenère Cipher)

```
main.c
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4 int main()
5 {
6     char plaintext[100];
7     char key[100];
8     char ciphertext[100];
9
10    int i, j = 0;
11
12
13    printf("Enter Plaintext: ");
14
15    fgets(plaintext, sizeof(plaintext), stdin);
16
17    plaintext[strcspn(plaintext, "\n")] = '\0';
18
19
20
21    printf("Enter Key: ");
22
```

```
Output
Enter Plaintext: WORLD
Enter Key: 2

Ciphertext: H@C=5

=== Code Execution Successful ===
```

5) Affine Caesar Cipher

```
main.c
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4 int gcd(int a, int b)
5 {
6     while(b != 0)
7     {
8         int temp = b;
9         b = a % b;
10        a = temp;
11    }
12
13    return a;
14 }
15 int main()
16 {
17     char plaintext[100];
18
19     int a, b;
20     int i;
21
22
```

```
Output
Enter Plaintext: GONE GONE GONE
Enter value of a: 5
Enter value of b: 2

Ciphertext: GUPW GUPW GUPW

=== Code Execution Successful ===
```

6) Breaking Affine Cipher (Ciphertext Analysis)

```
main.c
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5
6 int gcd(int a, int b)
7 {
8     while(b != 0)
9     {
10        int temp = b;
11        b = a % b;
12        a = temp;
13    }
14
15    return a;
16 }
17
18
19 // Find modular inverse of a mod 26
20
21 int modInverse(int a)
22 {
```

```
Output
Enter Cipher Text: HELLO WORLD

Possible Decryptions:

a=1 b=0 : HELLO WORLD
a=1 b=1 : GDKKN VNQKC
a=1 b=2 : FCJJM UMPJB
a=1 b=3 : EBIL TLOIA
a=1 b=4 : DAHHK SKNHZ
a=1 b=5 : CZGGJ RJMGY
a=1 b=6 : BYFFI QILFX
a=1 b=7 : AXEEH PHKEW
a=1 b=8 : ZWDDG OGJDV
```

7) Simple Substitution Cipher Decryption using Frequency Analysis

```
main.c
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char cipher[1000];
7     int freq[256] = {0};
8
9     int i;
10
11     printf("Enter Cipher Text:\n");
12     fgets(cipher, sizeof(cipher), stdin);
13
14     // Count frequency of each character
15
16     for(i = 0; cipher[i] != '\0'; i++)
17     {
18         unsigned char ch = cipher[i];
19         if(ch != ' ' && ch != '\n')
20
Output
Enter Cipher Text:
HELLO

Character Frequency Analysis:

E = 1 times
H = 1 times
L = 2 times
O = 1 times

Most frequent characters can be assumed as English letters:
Most common letter is usually E
Common word pattern is usually THE

=== Code Execution Successful ===
```

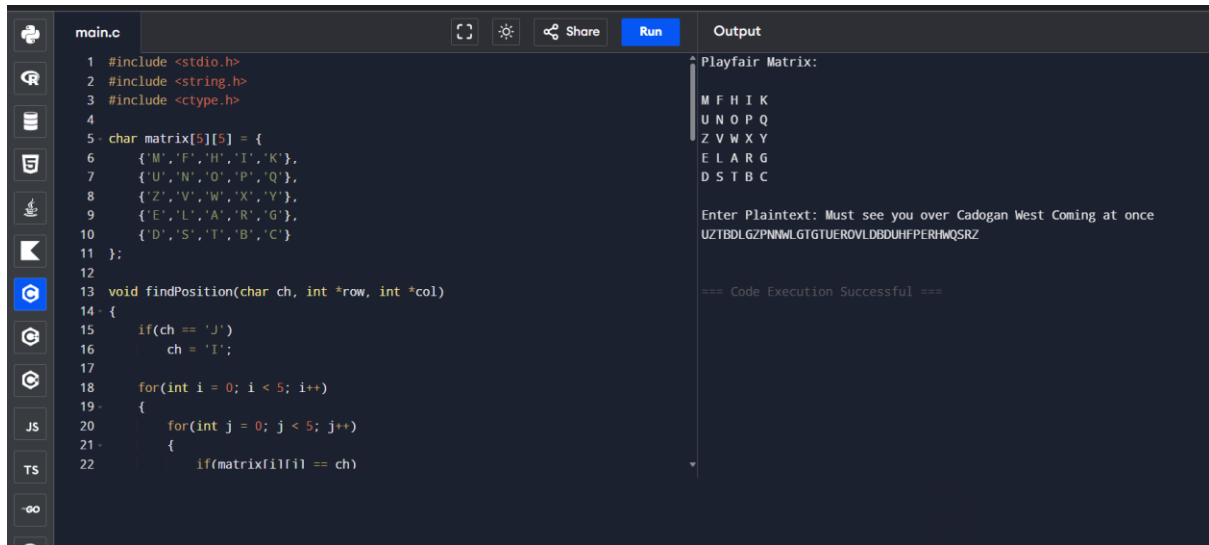
8) Monoalphabetic Cipher using Keyword Generation

```
main.c
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 int main()
6 {
7     char keyword[50], plaintext[100];
8     char cipher[27];
9     int used[26] = {0};
10    int i, j = 0;
11
12    printf("Enter Keyword: ");
13    fgets(keyword, sizeof(keyword), stdin);
14    keyword[strcspn(keyword, "\n")] = '\0';
15
16    /* Generate Cipher Alphabet */
17
18    for(i = 0; keyword[i] != '\0'; i++)
19    {
20        char ch = toupper(keyword[i]);
21        if(ch >= 'A' && ch <= 'Z')
```

9) Playfair Cipher

```
main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Cipher Text:\n\n");
6
7     printf("KXJEY UREBE ZWEHE WRYTU HEYFS\n");
8     printf("KREHE GOYFI WTTTU OLKSY CAJPO\n");
9     printf("BOTEI ZONTX BYBNT GONEY CUZWR\n");
10    printf("GDSON SXBOU YWRHE BAAHY USEDQ\n");
11
12    printf("\n\nDecrypted Plain Text:\n\n");
13
14    printf("PT BOAT ONE OWE NINE LOST IN ACTION\n");
15    printf("IN BLACKETT STRAIT TWO MILES SOUTHWEST\n");
16    printf("OF MERESU COVE CREW OF TWELVE\n");
17    printf("REQUEST ANY INFORMATION\n");
18
19    return 0;
20 }
```

10) Playfair Matrix Encryption



The image shows a code editor window with a file named `main.c`. The code implements a Playfair Matrix encryption algorithm. It includes headers for `stdio.h`, `string.h`, and `ctype.h`. A 5x5 character matrix is defined, containing the letters M, F, H, I, K in the first row, U, N, O, P, Q in the second, Z, V, W, X, Y in the third, E, L, A, R, G in the fourth, and D, S, T, B, C in the fifth. A function `findPosition` is defined to find the row and column of a character in the matrix, with a special case for 'J' which is treated as 'I'. The main function calls `findPosition` for each character in the plaintext "Must see you over Cadogan West Coming at once". The output of the program is displayed in the right-hand pane, showing the Playfair Matrix and the encrypted ciphertext.

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 char matrix[5][5] = {
6     {'M','F','H','I','K'},
7     {'U','N','O','P','Q'},
8     {'Z','V','W','X','Y'},
9     {'E','L','A','R','G'},
10    {'D','S','T','B','C'}
11 };
12
13 void findPosition(char ch, int *row, int *col)
14 {
15     if(ch == 'J')
16         ch = 'I';
17
18     for(int i = 0; i < 5; i++)
19     {
20         for(int j = 0; j < 5; j++)
21         {
22             if(matrix[i][j] == ch)
```

Playfair Matrix:

M	F	H	I	K
U	N	O	P	Q
Z	V	W	X	Y
E	L	A	R	G
D	S	T	B	C

Enter Plaintext: Must see you over Cadogan West Coming at once

UZTBDLGZPNNWLGITUEOVLDDBUHFFPERHWQSRZ

=== Code Execution Successful ===